CSP-J 第二轮模拟题

一．题目概况

中文题目名称	名字	点赞	摆渡车	对称二叉树
英文题目与子目录名	name	like	bus	tree
可执行文件名	name	like	bus	tree
输入文件名	name.in	like.in	bus.in	tree.in
输出文件名	name.out	like.out	bus.out	tree.out
每个测试点时限	1 秒	1 秒	2 秒	1 秒
测试点数目	20	25	20	25
每个测试点分值	5	4	5	4
附加样例文件	有	有	有	有
结果比较方式	全文比较（过滤行末空格及文末回车）			
题目类型	传统	传统	传统	传统
运行内存上限	256M	256M	256M	256M

二．提交源程序文件名

对于 C++语言	name.cpp	like.cpp	bus.cpp	tree.cpp
对于 C 语言	name.c	like.c	bus.c	tree.c
对于 pascal 语言	name.pas	like.pas	bus.pas	tree.pas

三．编译命令（不包含任何优化开关）

对于 C++语言	g++ -o name name.cpp -lm	g++ -o like like.cpp -lm	g++ -o bus bus.cpp -lm	g++ -o tree tree.cpp -lm

注意事项：

1、文件名（程序名和输入输出文件名）必须使用英文小写。

2、C/C++中函数 main()的返回值类型必须是 int，程序正常结束时的返回值必须是 0。

3、全国统一评测时采用的机器配置为： Intel(R) Core(TM) i7-8700K CPU @ 3.70GHz，内存 32GB。上述时限以此配置为准。

4、只提供 Linux 格式附加样例文件。

5、特别提醒：评测在当前最新公布的 NOI Linux 下进行，各语言的编译器版本以其为准。

1. 名字

(name.cpp/c/pas)

【问题描述】

单词是由大写或小写字母组成的字符串。名字是 首字母为大写字母 的单词(除首字母外，其他字母可以是大写也可以是小写)。

给定 N 个句子， 请你统计每个句子中分别有多少个名字。

【输入格式】

输入文件名为 name.in。

第一行包含一个正整数 N，表示句子的个数。

接下来 N 行输入 N 个句子。每个句子的字符总数不会超过 2 万。

【输出格式】

输出文件名为 name.out。

N 行，每行一个整数，表示这个句子里有几个名字。

【输入输出样例 1】

见选手目录下的 name / name1.in 和 name / name1.ans。

name.in	name.out
2 Fijswurma Yewjz hsi Zilwikb eaabulzzt	3 0

【输入输出样例 2】

见选手目录下的 name / name2.in 和 name / name2.ans。

name.in	name.out
3 Afujfcve tqnwkhk Ywaa vircqa jorrpskwi r Wmajguhd imfkdmp Cc	2 0 2

【输入输出样例 3】

见选手目录下的 name / name3.in 和 name / name3.ans。

【数据说明】

对于 100% 的数据， 1<=N<=1000

2. 点赞

（like.cpp/c/pas）

【问题描述】

小明维护着一个程序员论坛。现在他收集了一份"点赞"日志，日志共有M行。其中每一行的格式是：ts id，表示在ts时刻编号id的帖子收到一个"赞"。

现在小明想统计有哪些帖子曾经是"热帖"。如果一个帖子曾在任意一个长度为D的时间段内收到不少于K个赞，小明就认为这个帖子曾是"热帖"。

具体来说，如果存在某个时刻T满足该帖在长度为D的区间内收到不少于K个赞，该帖就曾是"热帖"。

给定日志，请你帮助小明统计出所有曾是"热帖"的帖子编号,从小到大输出。

【输入格式】

输入文件名为 like.in。

第一行包含三个整数N、M, D和K。 其中N表示帖子的个数，M表示日志的条数，D表示区间长度，K表示热帖的条件。

以下M行每行一条日志，包含两个整数ts和id。

【输出格式】

输出文件名为 like.out。

若干行，从小到大输出曾是热帖的帖子的编号。

【输入输出样例 1】

like.in	like.out
2 3 10 2 10 1 11 2 12 1	1

见选手目录下的 like/like1.in 和 like/like1.ans。

【输入输出样例 2】

like.in	like.out
10 7 10 2 0 1 0 10 10 10 10 1 9 1 100 3 100 3	1 3

见选手目录下的 like/like2.in 和 like/like2.ans。

【输入输出样例 3】

见选手目录下的 like/like3.in 和 like/like3.ans。

【数据说明】

对于40%的数据，1 <= K <= M <= 100， 0 <= D, ts <= 100，1 <= N <= 100

对于60%的数据，1 <= K <= M <= 1000 ， 0 <= D, ts <= 1000 ，1 <= N <= 1000

对于100%的数据，1 <= K <= M <= 100000， 0 <= D, ts <= 100000， 1 <= N <= 100000

pdfelement

3．摆渡车

(bus.cpp/c/pas)

【问题描述】

有 n 名同学要乘坐摆渡车从人大附中前往人民大学，第 i 位同学在第 t_i 分钟去等车。只有一辆摆渡车在工作，但摆渡车容量可以视为无限大。摆渡车从人大附中出发、把车上的同学送到人民大学、再回到人大附中（去接其他同学），这样往返一趟总共花费 m 分钟（同学上下车时间忽略不计）。摆渡车要将所有同学都送到人民大学。

凯凯很好奇，如果他能任意安排摆渡车出发的时间，那么这些同学的等车时间之和最小为多少呢？

注意：摆渡车回到人大附中后可以即刻出发。

【输入格式】

输入文件名为 bus.in。

第一行包含两个正整数 n，m，以一个空格分开，分别代表等车人数和摆渡车往返一趟的时间。

第二行包含 n 个正整数，相邻两数之间以一个空格分隔，第 i 个非负整数 t_i 代表第 i 个同学到达车站的时刻。

【输出格式】

输出文件名为 bus.out。

输出一行，一个整数，表示所有同学等车时间之和的最小值（单位：分钟）。

【输入输出样例 1】

bus.in	bus.out
5 1 3 4 4 3 5	0

见选手目录下的 bus/bus1.in 和 bus/bus1.ans。

【输入输出样例 1 说明】

同学 1 和同学 4 在第 3 分钟开始等车，等待 0 分钟，在第 3 分钟乘坐摆渡车出发。摆渡车在第 4 分钟回到人大附中。

同学 2 和同学 3 在第 4 分钟开始等车，等待 0 分钟，在第 4 分钟乘坐摆渡车出发。摆渡车在第 5 分钟回到人大附中。

同学 5 在第 5 分钟开始等车，等待 0 分钟，在第 5 分钟乘坐摆渡车出发。自此所有同学都被送到人民大学。总等待时间为 0。

【输入输出样例 2】

bus.in	bus.out
5 5 11 13 1 5 5	4

见选手目录下的 bus/bus2.in 和 bus/bus2.ans。

【输入输出样例 2 说明】

同学 3 在第 1 分钟开始等车，等待 0 分钟，在第 1 分钟乘坐摆渡车出发。摆渡车在第 6 分钟回到人大附中。

同学 4 和同学 5 在第 5 分钟开始等车，等待 1 分钟，在第 6 分钟乘坐摆渡车出发。摆渡车在第 11 分钟回到人大附中。

同学 1 在第 11 分钟开始等车，等待 2 分钟；同学 2 在第 13 分钟开始等车，等待 0 分钟。他/她们在第 13 分钟乘坐摆渡车出发。自此所有同学都被送到人民大学。总等待时间为 4。可以证明，没有总等待时间小于 4 的方案。

【输入输出样例 3】

见选手目录下的 bus/bus3.in 和 bus/bus3.ans。

【数据规模与约定】

对于 10% 的数据，$n \leq 10,\ m = 1,\ 0 \leq t_i \leq 100$。

对于 30% 的数据，$n \leq 20,\ m \leq 2,\ 0 \leq t_i \leq 100$。

对于 50% 的数据，$n \leq 500,\ m \leq 100,\ 0 \leq t_i \leq 10^4$。

另有 20% 的数据，$n \leq 500,\ m \leq 10,\ 0 \leq t_i \leq 4 \times 10^6$。

对于 100% 的数据，$n \leq 500,\ m \leq 100,\ 0 \leq t_i \leq 4 \times 10^6$。

4．对称二叉树

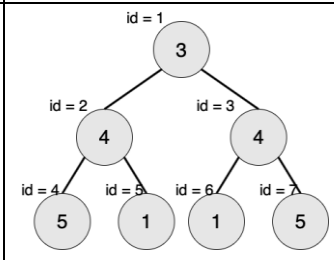
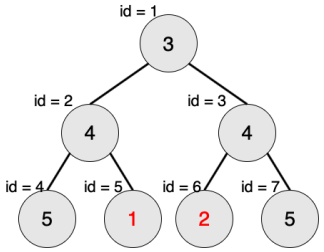
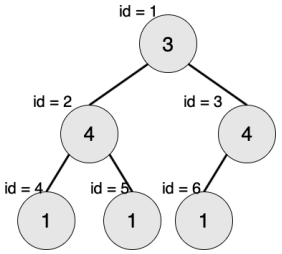
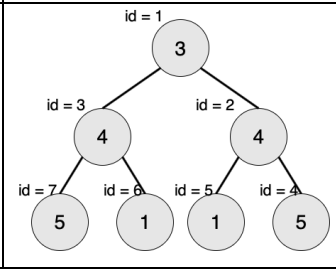
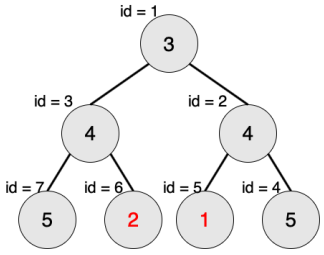
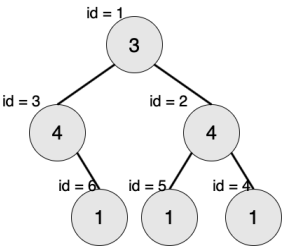
(tree.cpp/c/pas)

【问题描述】

一棵有点权的有根树如果满足以下条件，则被轩轩称为**对称二叉树**：

1. 二叉树；
2. 将这棵树**所有**节点的左右子树交换，新树和原树对应位置的结构相同且点权相等。

下图中节点内的数字为权值，节点外的 id 表示节点编号。

	对称二叉树	非对称二叉树 （权值不对称）	非对称二叉树 （结构不对称）
原树			
所有节点的左右子树交换后			

现在给出一棵二叉树，希望你找出它的一棵子树，**该子树为对称二叉树，且节点数最多**。请输出这棵子树的节点数。

注意：只有树根的树也是对称二叉树。本题中约定，以节点 T 为子树根的一棵“子树”指的是：节点 T 和它的**全部**后代节点构成的二叉树。

【输入格式】

输入文件名为 `tree.in`。

第一行一个正整数 n，表示给定的树的节点的数目，规定节点编号 1~n，其中节点 1 是树根。

第二行 n 个正整数，用一个空格分隔，第 i 个正整数 v_i 代表节点 i 的权值。

接下来 n 行，每行两个正整数 l_i, r_i，分别表示节点 i 的左右孩子的编号。如果不存在左 / 右孩子，则以 -1 表示。两个数之间用一个空格隔开。

【输出格式】

输出文件名为 `tree.out`。

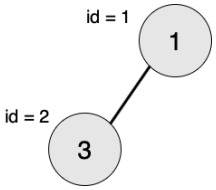
输出文件共一行，包含一个整数，表示给定的树的最大对称二叉子树的节点数。

【输入输出样例 1】

tree.in	tree.out
2 1 3 2 -1 -1 -1	1

见选手目录下的 tree/tree1.in 和 tree/tree1.ans。

【输入输出样例 1 说明】



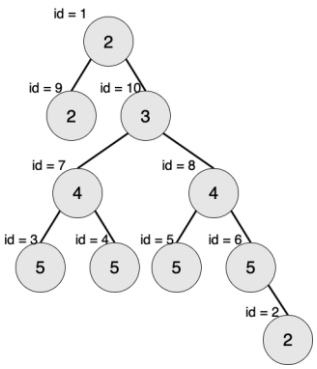
最大的对称二叉子树为以节点 2 为树根的子树，节点数为 1。

【输入输出样例 2】

tree.in	tree.out
10 2 2 5 5 5 5 4 4 2 3 9 10 -1 -1 -1 -1 -1 -1 -1 -1 -1 2 3 4 5 6 -1 -1 7 8	3

见选手目录下的 tree/tree2.in 和 tree/tree2.ans。

【输入输出样例 2 说明】



最大的对称二叉子树为以节点 7 为树根的子树，节点数为 3。

【输入输出样例 3】

见选手目录下的 tree/tree3.in 和 tree/tree3.ans。

【数据规模与约定】

共 25 个测试点。

$v_i \leq 1000$。

测试点 1~3，$n \leq 10$，保证根结点的左子树的所有节点都没有右孩子，根结点的右子树的所有节点都没有左孩子。

测试点 4~8，$n \leq 10$。

测试点 9~12，$n \leq 10^5$，保证输入是一棵“满二叉树”。

测试点 13~16，$n \leq 10^5$，保证输入是一棵“完全二叉树”。

测试点 17~20，$n \leq 10^5$，保证输入的树的点权均为 1。

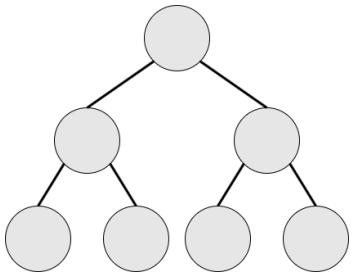
测试点 21~25，$n \leq 10^6$。

本题约定：

层次：节点的层次从根开始定义起，根为第一层，根的孩子为第二层。树中任一节点的层次等于其父亲节点的层次加 1。

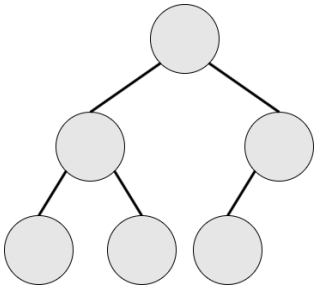
树的深度：树中节点的最大层次称为树的深度。

满二叉树：设二叉树的深度为 h，且二叉树有 $2^h - 1$ 个节点，这就是满二叉树。

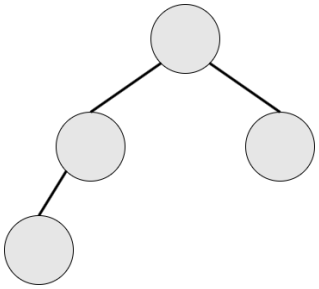


满二叉树（深度为 3）

完全二叉树：设二叉树的深度为 h，除第 h 层外，其它各层的结点数都达到最大个数，第 h 层所有的结点都连续集中在最左边，这就是完全二叉树。



完全二叉树（深度为 3）



完全二叉树（深度为 3）